

0.1 Sum-Check-based Grand Product Protocol

This subsection describes a transparent SNARK, which may be used for proving grand product relations of the following form:

$$\mathcal{R}_{GP} = \left\{ (p \in \mathbb{F}, \mathbf{v} \in \mathbb{F}^m) : p = \prod_{i=0}^m v_i \right\} \quad (1)$$

Without loss of generality, assume that m is a power of 2. We can think of $\mathbf{v}$ as a vector of evaluations of a $\log m$ -variate multilinear polynomial $v(x)$ over $\{0, 1\}^{\log m}$ in a natural fashion. We assume the prover first opens p and then commits to $v(x)$; the following protocol then verifies both the polynomial's validity and the corresponding grand-product equality derived from that commitment.

Lemma 0.1. A scalar p and a vector $\mathbf{v}$ satisfies the relation $\mathcal{R}_{GP}$ if and only if there exists a multilinear polynomial f in $\log m + 1$ variables such that $f(1, \dots, 1, 0) = p$ and $\forall x \in \{0, 1\}^{\log m}$ the following hold:

$$\begin{aligned} f(0, x) &= v(x) \\ f(1, x) &= f(x, 0) \cdot f(x, 1) \end{aligned}$$

Such polynomial f has the following construction:

- $f(1, \dots, 1) = 0$
- For all $\ell \in [\log m]$ and $x \in \{0, 1\}^{\log m - \ell}$:

$$f(1^\ell, 0, x) = \prod_{y \in \{0, 1\}^\ell} v(x, y)$$

One can simply visualize the structure of f as a binary tree.

Example 0.1. Let $m = 4$ ($\log m = 2$), $\mathbf{v} = \{1, 2, 3, 4\}$, consequently $p = 1 \times 2 \times 3 \times 4 = 24$, then:

$$\begin{aligned} v(x_1, x_2) &= 1 + 2x_1 + x_2 \\ v(x_1, x_2) : \quad &v(0, 0) = 1, \quad v(0, 1) = 2, \quad v(1, 0) = 3, \quad v(1, 1) = 4. \end{aligned}$$

Now, we define f as follows:

$$f(0, 0, 0) = 1, \quad f(0, 0, 1) = 2, \quad f(0, 1, 0) = 3, \quad f(0, 1, 1) = 4,$$

and:

$$\begin{aligned} f(1, 0, 0) &= f(0, 0, 0) \times f(0, 0, 1) = 1 \times 2 = 2, \\ f(1, 0, 1) &= f(0, 1, 0) \times f(0, 1, 1) = 3 \times 4 = 12, \\ f(1, 1, 0) &= f(1, 0, 0) \times f(1, 0, 1) = 2 \times 12 = 24 = p, \\ f(1, 1, 1) &= 0. \end{aligned}$$

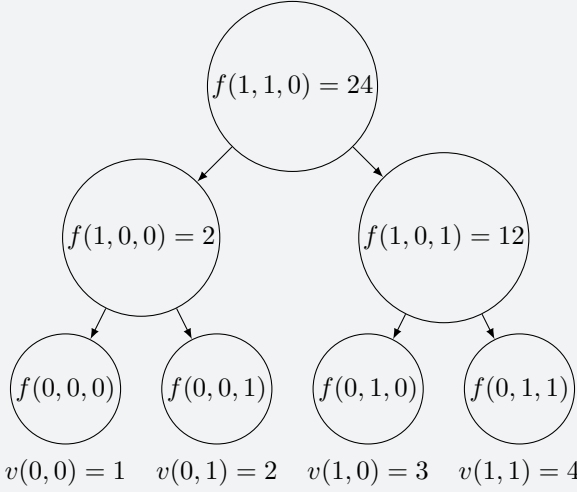


Illustration: Binary tree of the Grand Product constraints for $v(x_1, x_2) = 1 + 2x_1 + x_2$.

Then, to check that $\forall x \in \{0, 1\}^{\log m}$ the equation $f(1, x) = f(x, 0) \cdot f(x, 1)$ holds, we can use a sum-check protocol to prove the evaluation of g that is referred to a MLE of $f(1, x) - f(x, 0) \cdot f(x, 1)$:

$$g(t) = \sum_{x \in \{0, 1\}^{\log m}} \tilde{\text{eq}}(t, x) \cdot (f(1, x) - f(x, 0) \cdot f(x, 1))$$

By the Schwartz–Zippel lemma, except for a soundness error of $\frac{\log m}{|\mathbb{F}|}$ (which should be negligible), $g(\tau) = 0$ for τ uniformly random in $\mathbb{F}^{\log m}$ if and only if $g = 0$, which implies that $f(1, x) - f(x, 0) \cdot f(x, 1) = 0$ for all $x \in \{0, 1\}^{\log m}$.

Similarly, to prove that $v(x) = f(0, x)$ for all $x \in \{0, 1\}^{\log m}$ it suffices to prove that $v(\gamma) = f(0, \gamma)$ for a public coin $\gamma \in \mathbb{F}^{\log m}$.

Thus, to prove the existence of f and hence the grand product relationship, it suffices to prove, for some verifier selected random $\tau, \gamma \in \mathbb{F}^\ell$, that:

$$0 = \sum_{x \in \{0, 1\}^{\log m}} \tilde{\text{eq}}(x, \tau) \cdot (f(1, x) - f(x, 0) \cdot f(x, 1)) \quad (2)$$

$$f(0, \gamma) = v(\gamma) \quad (3)$$

$$f(1, \dots, 1, 0) = p \quad (4)$$

This can be achieved by running the sum-check protocol between $\mathcal{P}$ and $\mathcal{V}$, where $\mathcal{V}$ has oracle access to v and f . Additionally, $\mathcal{V}$ evaluates the f, v at the random point γ to verify that $f(0, x) = v(x)$. So, the sum-check-based protocol for the grand products looks as follows:

Algorithm 1: Sum-Check-based Protocol for Grand Products

1 $\mathcal{P}$: Compute polynomials $v \in \mathbb{F}^{\log m}[x]$, $f \in \mathbb{F}^{\log m+1}[x]$ such that

$$p = \prod_{x \in \{0,1\}^{\log m}} v(x) \quad \text{and} \quad f, v \text{ satisfy (2), (3), (4).}$$

2 $\mathcal{P}$: $C_f \leftarrow \text{Commit}(f)$; $C_v \leftarrow \text{Commit}(v)$; send C_f, C_v to $\mathcal{V}$.

3 $\mathcal{V}$: Choose random $\tau, \gamma \in \mathbb{F}^{\log m}$ and send them to $\mathcal{P}$.

4 $\mathcal{P}$: Compute

$$g(x) = \tilde{\text{eq}}(x, \tau) (f(1, x) - f(x, 0) f(x, 1)).$$

5 $\mathcal{P} \ \& \ \mathcal{V}$: Run $\text{SumCheckProtocol}(0, g, C_f)$

/* we assume this sum-check runs over the commitment to f , not
directly on g . */

6 $\mathcal{V}$: $r \leftarrow \text{Query}(C_f, (1, \dots, 1, 0))$.

7 **if** $r \neq p$ **then**

8 | $\mathcal{V}$ **rejects**.

9 **end**

10 $\mathcal{V}$: $a \leftarrow \text{Query}(C_f, (0, \gamma))$, $v(\gamma) \leftarrow \text{Query}(C_v, \gamma)$.

11 **if** $a \neq v(\gamma)$ **then**

12 | $\mathcal{V}$ **rejects**.

13 **end**

Remark. Note, that during the call to the sum-check protocol, we pass the polynomial g in a couple with the commitment to f . Here we assume that each invocation of the `Query` method to g inside the sum-check instance will be more complex: **instead of evaluating a single function and returning result, we evaluate f at three different points, check the proofs and return results according to the construction of g .**

0.2 Randomized Permutation Check

The goal of the randomized permutation check is to verify, with high probability, whether two sequences of tuples are permutations of each other, without performing a full sort or pairwise comparison.

Definition 0.2 (Reed-Solomon Fingerprinting). Let $\mathbf{a} \in \mathbb{F}^n$, then for a random $\gamma \in \mathbb{F}$, the Reed-Solomon fingerprinting of $\mathbf{a}$ is defined as:

$$h_\gamma(\mathbf{a}) = \sum_{i \in [n]} a_i \cdot \gamma^i.$$

$h_\gamma(\mathbf{a})$ uniquely identifies the sequence $\mathbf{a}$ with high probability, i.e., let $\mathbf{b} \in \mathbb{F}^n$

and $\mathbf{a} \neq \mathbf{b}$, then, according to the Schwartz-Zippel lemma:

$$\Pr[h_\gamma(\mathbf{a}) = h_\gamma(\mathbf{b})] \leq \frac{n}{|\mathbb{F}|}.$$

Definition 0.3 (Randomized Permutation Check). Let A and B be two multisets of tuples in $\mathbb{F}^n$. Define

$$\mathcal{H}_{\tau, \gamma}(X) = \prod_{x \in X} (h_\gamma(x) - \tau).$$

Then comparing $\mathcal{H}_{\tau, \gamma}(A)$ and $\mathcal{H}_{\tau, \gamma}(B)$ yields a randomized test for whether A and B are permutations of one another. Concretely:

- (Completeness) If $A = B$ (as multisets), then

$$\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)$$

with probability 1 over uniform $\tau, \gamma \in \mathbb{F}$.

- (Soundness) If $A \neq B$, then

$$\Pr[\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)] \leq \frac{\max(|A|, |B|)}{|\mathbb{F}|}.$$

In other words, by first “hashing” each tuple via the map function h_γ and then taking the τ -shifted product, one obtains a fingerprint that is invariant under permutation but unlikely to collide on distinct sets.

Example 0.2. Consider all operations in $\mathbb{F}_7$, and set $n = 3$, $\tau = 5$, and $\gamma = 3$. Let

$$A = \{(1, 2, 3), (4, 0, 6)\}, \quad B = \{(4, 0, 6), (1, 2, 3)\},$$

First compute the Reed-Solomon fingerprints modulo 7:

$$h_\gamma(1, 2, 3) = 1 \cdot 3^0 + 2 \cdot 3^1 + 3 \cdot 3^2 = 1 + 6 + 6 = 13 \equiv 6,$$

$$h_\gamma(4, 0, 6) = 4 \cdot 1 + 0 \cdot 3 + 6 \cdot 2 = 4 + 0 + 12 = 16 \equiv 2.$$

Now form the shifted products:

$$\mathcal{H}_{\tau, \gamma}(A) = (6 - 5)(2 - 5) = 1 \cdot (-3) \equiv 4,$$

$$\mathcal{H}_{\tau, \gamma}(B) = (2 - 5)(6 - 5) = (-3) \cdot 1 \equiv 4,$$

so the test *accepts* A vs. B (they are indeed permutations).

Now consider a non-permutation B' :

$$B' = \{(1, 2, 3), (2, 1, 3)\}.$$

$$h_\gamma(2, 1, 3) = 2 \cdot 1 + 1 \cdot 3 + 3 \cdot 2 = 2 + 3 + 6 = 11 \equiv 4.$$

$$\mathcal{H}_{\tau, \gamma}(B') = (6 - 5)(4 - 5) = 1 \cdot (-1) \equiv 6 \neq 4,$$

so the test *rejects* A vs. B' . In this way a single choice of (γ, τ) distinguishes permutations from non-permutations.

0.3 Offline Memory Checking

This protocol lets you, after doing a bunch of read/writes from an untrusted memory, verify in one quick algebraic step that every answer really came from the same original contents – without re-running any reads.

For the first time it's not so obvious, why we may need need such a protocol, so let's start with a simple example.

Example 0.3. Consider Alice, who stores two values on Bob's dedicated server at addresses 0 and 1. Initially, Bob's memory contains

$$M = \{(0, 100), (1, 200)\}.$$

Alice then performs the following operations in sequence:

1. Alice writes a new value 150 at address 0. Bob updates his memory to $\{(0, 150), (1, 200)\}$.
2. Alice reads from address 0 and obtains the reply 150.
3. Alice reads from address 1 and (honestly) obtains 200.

At this point, both replies individually look correct. However, Bob can cheat on the very last step by returning, say, 300 instead of 200:

$$(1, 200) \longrightarrow (1, 300).$$

Alice only sees the single read result “300 at address 1,” which could simply be the true value after some write she missed—and she has no immediate way to detect the inconsistency.

In other words, without keeping an auditable record of all reads, Alice cannot later prove that all replies came from a single, immutable memory state. This gap is exactly what an *offline memory check* fills: after all reads are done, Alice runs one fast verification that guarantees “every read you saw really did come from one fixed initial memory + your writes,” and thus catches any cheating by Bob.

Each memory cell can be described as a tuple $(\text{addr}, \text{val}, \text{counter})$, where addr is the address of the memory cell, val is the value stored at that address, and counter is the number of times the memory cell has been accessed. You can think of a counter as a timestamp. The memory is a vector of memory cels.

The protocol utilizes four sets of tuples:

- **init** – contains the initial memory state, where all counters are set to 0;
- **write** – contains memory cels that represent write operations, where *val* is the value stored at the memory address *addr* after the specified counter;
- **read** – contains memory cels that represent read operations, where *val* is the value read from the memory address *addr* at the specified counter;
- **final** – contains the final memory state, where all counters are set to the last value after the last read/write operation.

At the beginning, **init** is populated with the initial memory state, where all counters are set to 0, indicating that the values were first accessed during the initialization phase, while the **read**, **write** are empty. Then, for each read operation, the untrusted memory is queried at *addr*, returning a pair (*val*, *counter*), and writing a tuple (*addr*, *val*, *counter*) to the **read** set. For each write operation a corresponding tuple (*addr*, *newval*, *counter* + 1), where *newval* is a value being written, is added, with *counter* incremented by one, to the **write** set.

We assume that before each write operation, the read operation is performed. And the read operation is always followed by a write operation to the same address. Thus, to make a plain read operation, the prover first reads the value from the memory, then writes it back to the memory.

After all reads and writes are done, the **final** set is populated with the final memory state. Now, the verifier can check the consistency of all the operations by verifying the following equation:

$$\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}.$$

Thus, using randomized permutation check, one can rewrite the equation as follows:

$$\begin{aligned} \mathcal{H}_{\tau,\gamma}(\mathbf{read}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{final}) &= \mathcal{H}_{\tau,\gamma}(\mathbf{write}) \cdot \mathcal{H}_{\tau,\gamma}(\mathbf{init}), \\ \prod_{(a,v,t) \in \mathbf{read}} (h_{\gamma}(a,v,t) - \tau) &\prod_{(a,v,t) \in \mathbf{final}} (h_{\gamma}(a,v,t) - \tau) = \\ = \prod_{(a,v,t) \in \mathbf{write}} (h_{\gamma}(a,v,t) - \tau) &\prod_{(a,v,t) \in \mathbf{init}} (h_{\gamma}(a,v,t) - \tau), \\ \prod_{(a,v,t) \in \mathbf{read}} ((a\gamma^2 + v\gamma + t) - \tau) &\prod_{(a,v,t) \in \mathbf{final}} ((a\gamma^2 + v\gamma + t) - \tau) = \\ = \prod_{(a,v,t) \in \mathbf{write}} ((a\gamma^2 + v\gamma + t) - \tau) &\prod_{(a,v,t) \in \mathbf{init}} ((a\gamma^2 + v\gamma + t) - \tau). \end{aligned}$$

Example 0.4. Suppose the we track a single *row-memory*, the algorithm performs three reads and two writes, the initial memory is given by the following table:

Address	0	1	2	3
Value	2	5	7	9

Initial memory (counter 0):

$$\mathbf{init} = \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\},$$

while

$$\mathbf{read} = \emptyset \quad \text{and} \quad \mathbf{write} = \emptyset.$$

Trace 1 - consistent execution

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ (5, 0)	(1, 5, 0)	-
2	write((1,6))	-	(1, 6, 1)
3	read(2) $\rightarrow$ (7, 0)	(2, 7, 0)	-
4	write((2,7))	-	(2, 7, 1)

After the four steps

$$\mathbf{read} = \{(1, 5, 0), (2, 7, 0)\},$$

$$\mathbf{write} = \{(1, 6, 1), (2, 7, 1)\}.$$

And we can fill **final** as:

$$\mathbf{final} = \{(0, 2, 0), (1, 6, 1), (2, 7, 1), (3, 9, 0)\}$$

One can clearly see that $\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}$.

$$\begin{aligned} & \{(1, 5, 0), (2, 7, 0)\} \cup \{(0, 2, 0), (1, 6, 1), (2, 7, 1), (3, 9, 0)\} = \\ & = \{(1, 6, 1), (2, 7, 1)\} \cup \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\} \end{aligned}$$

Trace 2 - inconsistent execution

Let the second reading return a wrong value – a , instead of 5. So, the new trace would look like:

step	operation	$\Delta\mathbf{read}_{\text{step}}$	$\Delta\mathbf{write}_{\text{step}}$
1	read(1) $\rightarrow$ (a , 0)	(1, a , 0)	-
2	write((1,6))	-	(1, 6, 1)

After the two steps

$$\mathbf{read} = \{(1, a, 0)\},$$

$$\mathbf{write} = \{(1, 6, 1)\},$$

The verifier checks $\mathbf{read} \cup \mathbf{final} = \mathbf{write} \cup \mathbf{init}$:

$$\begin{aligned} & \{(1, a, 0)\} \cup \{(0, 2, 0), (1, 6, 1), (2, 7, 0), (3, 9, 0)\} \neq \\ & \neq \{(1, 6, 1)\} \cup \{(0, 2, 0), (1, 5, 0), (2, 7, 0), (3, 9, 0)\} \end{aligned}$$

and *rejects*.

Actually, no multiset **final** that depends on the valid values and addresses can reconcile them.